QuestionsLab2.2.pdf

abcd__1234

Paralelismo

3º Grado en Ingeniería Informática

Facultad de Informática de Barcelona (Fib)
Universidad Politécnica de Catalunya

Accede al documento original

Qualificació **8,00** sobre 8,00 (**100**%)

Pregunta 1

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

Regarding the pi versions v9, v10, v11, v12, v13 and v14. Which of the following sentences are true:

- a. Versions 11 and 12 cannot achieve speedups larger than 2 because they have too much overhead due to task synchronization.
- b. Versions 11 and 12 use two tasks to distribute the work. On the other hand, version 11 is faster than 12 thanks to the use of the reduction clause.
- c. Versions 9 and 10 have data race conditions updating sum, but both of them can be corrected by adding a single construct before updating sum.
- d. Version 14 exploits the parallelism of the Pi program better because the granularity of the tasks is smaller and the cost of synchronization is reduced using the reduction clause.
- e. Version 13 has also two tasks with dependencies and does not use any task barrier inside the single. Dependencies help an extra task to wait for the two tasks doing the Pi computation. The last task is waited for at the end of the single clause.

Pregunta 2

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

Which of the following sentences are true (one or more):

- [ ] a. A nowait clause in a single construct of a code allows more than one thread to execute the code within this single construct without waiting.
- [x] b. Single construct only allows one thread to execute the code inside its context. The rest of the threads will wait at the end of the single construct.
- [ ] c. Single construct allows to avoid data race condition updating a shared sum variable. For instance, in the Pi program we can add a single construct before (sum +=) to exclusively increase sum by all the threads of the parallel region.
- [x] d. The code inside two consecutive single constructs in the same parallel region may be executed in parallel by two threads if the first single in the parallel region has a nowait clause.

Pregunta 3

Completa

Puntuació 1,00 sobre 1,00

Marqueu aquesta pregunta per a una referència futura

Assume we want to use explicit tasks in our OpenMP program. Which of the following sentences are true (one or more):

a. We don't need to create a parallel region to execute explicit tasks in parallel. Actually, the first thing we usually do is add a single construct after the parallel construct to allow only one thread to create tasks.

b. The number of independent tasks that can be concurrently executed depends on the number of threads of your parallel region and the overall number of tasks you have.

c. Assume a parallel region of 4 threads with a for loop of N iterations for(i=0;i<N;i++), variable i declared before the parallel region and the following loop body: #pragma omp task {printf("hola \%d\n",i);}. There will be N messages "hola i value" with i value going from 0 to N-1.

d. Assume a parallel region of 4 threads with a for loop of N iterations and the following loop body: #pragma omp task {printf("hola %d\n",i);}. There will be 4 x N messages "hola i value" with i value going from 0 to N-1 if the variable is private or firstprivate.

e. Assume a parallel region of 4 threads with a for loop of N iterations and the following body loop: #pragma omp task {printf("hola %d\n",i);} and variable i is shared. There will be a non-deterministic number of messages "hola i value" because all threads will run the same parallel region code, including the for and updating the same induction variable i.

Pregunta 4

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

Assume an OpenMP program using the taskloop construct. Which of the following sentences are correct (one or more):

- a. grainsize and num_tasks clauses can be used at the same time to specify the details of task granularity.
- b. grainsize(n) determines the number of consecutive iterations that should be assigned to each explicit task. However, the actual number of iterations that will be assigned may change depending on the global number of iterations and if it is a multiple of n.
- c. taskloop construct waits, at the end of the taskloop, only for the explicit tasks generated by it. However, if the programmer defines nested tasks inside they will not be waited for.
- d. Tasks generated in two consecutive taskloop, inside the same parallel and single region, cannot be executed at the same time except if the first taskloop includes the nogroup clause.

Pregunta 5

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

Assume 4.reduction program. Which of the following sentences are correct:

- a. It is possible to use a taskloop reduction that is included in a taskgroup with a task_reduction clause.
- b. We cannot use a reduction clause in a taskloop if it is included in a taskgroup with a task_reduction clause.
- c. Taskloop uses reduction to specify that all tasks created contribute to the reduction of a variable updated in the loop.
- d. There is no way to perform a reduction operation on a variable with explicit tasks.

Pregunta 6

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

Assume 5.synchtasks program. Which of the following sentences are correct:

- a. The Task Dependence Graph of the program consists of 5 tasks with the following characteristics: foo1 and foo2 have to be computed before foo4. foo3 can be done in parallel with foo1, foo2 and foo4. foo5 should wait for foo4 and foo3 to be finished. Finally, foo5 is synchronized at the end of the single.
- b. Without changing the order of creation of the program tasks or adding additional tasks, it is possible to achieve the same TDG by just adding taskwaits instead of using depend() clauses.
- c. Without changing the order of creation of the program tasks or adding additional tasks, it is possible to achieve the same TDG by just adding taskgroups instead of using depend() clauses.
- d. Assume we move up task foo3 before task foo1 is called. Then, we could have task foo3 running in parallel with a taskgroup including 1) another taskgroup of foo1 and foo2 and 2) task foo4, then a taskwait (waiting for foo3 and taskgroup) and finally task foo5. In this case, we will achieve the same TDG.

Pregunta 7

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

After submitting to execution pi_omp_parallel with one step and 32 threads, observing the result obtained we have that (select only one correct option):

- a. The total overhead time is incremented proportionally to the number of threads because the work is replicated.
- b. The overhead per thread is incremented as there are more threads to synchronize.
- c. The resulting overhead per thread decrements when incrementing the number of threads, till a value close to 1 microsecond.
- d. The showed overhead per thread becomes almost constant when incrementing the number of threads, decreasing very slightly in an almost asymptotic manner.

SN 25/26

PARA LA COMUNIDAD UNIVERSITARIA

EL SUBIDÓN

DE SIERRA NEVADA

Expert	Long expert
2 DÍAS CONSECUTIVOS	**5 DÍAS** DE LIBRE USO
86 €	**185 €**

Más info.: **elsubidon.es**

sierranevada
granada · Andalucía

Acaba la

Pregunta 8

Completa

Puntuació 1,00 sobre 1,00

Marca la pregunta

The results of the experiments indicated in the assignment related with the task creation overhead showed me that (select only one correct option):

a. The task creation overhead depends on the number of participating threads.

b. The total overhead corresponds only to the task creation; the overhead per task corresponds to task synchronization.

c. The overhead of task creation and synchronization is almost constant and close to a value of 0.1 microseconds.

d. The overhead of task creation and synchronization increments proportionally to the number of tasks.

Acaba la revisió